

5.5.1 Other LU factorization algorithms ¶

There are actually five different (unblocked) algorithms for computing the LU factorization that were discovered over the course of the centuries. Here we show how to systematically derive all five. For details, we suggest Week 6 of our Massive Open Online Course titled "LAFF-On Programming for Correctness" [29].

Remark 5.5.1.1. To put yourself in the right frame of mind, we highly recommend you spend about an hour reading the paper

- [32] Devangi N. Parikh, Margaret E. Myers, Richard Vuduc, Robert A. van de Geijn, A Simple Methodology for Computing Families of Algorithms, FLAME Working Note #87, The University of Texas at Austin, Department of Computer Science, Technical Report TR-18-06. [arXiv:1808.07832](https://arxiv.org/abs/1808.07832).

```

A = LU-var1(A)
A → ( ATL | ATR
      ABL | ABR )
ATL is 0 × 0
while n(ATL) < n(A)
    ( ATL | ATR
      ABL | ABR ) → ( A00 | a01 A02
                      a10T | α11 a12T
                      A20 | a21 A22 )
    ...
    ( ATL | ATR
      ABL | ABR ) ← ( A00 | a01 A02
                      a10T | α11 a12T
                      A20 | a21 A22 )
endwhile

```

Figure 5.5.1.2. LU factorization algorithm skeleton.

Finding the algorithms starts with the following observations.

- Our algorithms will overwrite the matrix A , and hence we introduce $\hat{A}$ to denote the original contents of A . We will say that the precondition for the algorithm is that

$$A = \hat{A}$$

(A starts by containing the original contents of A .)

- We wish to overwrite A with L and U . Thus, the postcondition for the algorithm (the state in which we wish to exit the algorithm) is that

$$A = L \setminus U \wedge LU = \hat{A}$$

(A is overwritten by L below the diagonal and U on and above the diagonal, where multiplying L and U yields the original matrix A .)

- All the algorithms will march through the matrices from top-left to bottom-right, giving us the code skeleton in Figure 5.5.1.2. Since the computed L and U overwrite A , throughout they are partitioned comformal to (in the same way) as is A .
- Thus, before and after each iteration of the loop the matrices are viewed as quadrants:

$$A \rightarrow \left(\begin{array}{c|c} A_{TL} & A_{TR} \\ \hline A_{BL} & A_{BR} \end{array} \right), L \rightarrow \left(\begin{array}{c|c} L_{TL} & 0 \\ \hline L_{BL} & L_{BR} \end{array} \right), \text{ and } U \rightarrow \left(\begin{array}{c|c} U_{TL} & U_{TR} \\ \hline 0 & U_{BR} \end{array} \right).$$

where A_{TL} , L_{TL} , and U_{TL} are all square and equally sized.

- In terms of these exposed quadrants, in the end we wish for matrix A to contain

$$\begin{aligned} \left(\begin{array}{c|c} A_{TL} & A_{TR} \\ \hline A_{BL} & A_{BR} \end{array} \right) &= \left(\begin{array}{c|c} L \setminus U_{TL} & U_{TR} \\ \hline L_{BL} & L \setminus U_{BR} \end{array} \right) \\ \wedge \left(\begin{array}{c|c} L_{TL} & 0 \\ \hline L_{BL} & L_{BR} \end{array} \right) \left(\begin{array}{c|c} U_{TL} & U_{TR} \\ \hline 0 & U_{BR} \end{array} \right) &= \left(\begin{array}{c|c} \hat{A}_{TL} & \hat{A}_{TR} \\ \hline \hat{A}_{BL} & \hat{A}_{BR} \end{array} \right) \end{aligned}$$

- Manipulating this yields what we call the Partitioned Matrix Expression (PME), which can be viewed as a recursive definition of the LU factorization:

$$\begin{aligned} \left(\begin{array}{c|c} A_{TL} & A_{TR} \\ \hline A_{BL} & A_{BR} \end{array} \right) &= \left(\begin{array}{c|c} L \setminus U_{TL} & U_{TR} \\ \hline L_{BL} & L \setminus U_{BR} \end{array} \right) \\ \wedge \frac{L_{TL}U_{TL} = \hat{A}_{TL} \quad L_{TL}U_{TR} = \hat{A}_{TR}}{L_{BL}U_{TL} = \hat{A}_{BL} \quad L_{BR}U_{BR} = \hat{A}_{BR} - L_{BL}U_{TR}} \end{aligned}$$

- Now, consider the code skeleton for the LU factorization in Figure 5.5.1.2. At the top of the loop (right after the **while**), we want to maintain certain contents in matrix A . Since we are in a loop, we haven't yet overwritten A with the final result. Instead, some progress toward this final result has been made. The way we can find what the state of A is that we would like to maintain is to take the PME and delete subexpressions. For example, consider the following condition on the contents of A :

$$\left(\begin{array}{c|c} A_{TL} & A_{TR} \\ \hline A_{BL} & A_{BR} \end{array} \right) = \left(\begin{array}{c|c} L \setminus U_{TL} & U_{TR} \\ \hline L_{BL} & \hat{A}_{BR} - L_{BL}U_{TR} \end{array} \right) \\ \wedge \frac{L_{TL}U_{TL} = \hat{A}_{TL} \quad L_{TL}U_{TR} = \hat{A}_{TR}}{L_{BL}U_{TL} = \hat{A}_{BL}}.$$

What we are saying is that A_{TL} , A_{TR} , and A_{BL} have been completely updated with the corresponding parts of L and U , and A_{BR} has been partially updated. This is exactly the state that the right-looking algorithm that we discussed in [Subsection 5.2.2](#) maintains! What is left is to factor A_{BR} , since it contains $\hat{A}_{BR} - L_{BL}U_{TR}$, and $\hat{A}_{BR} - L_{BL}U_{TR} = L_{BR}U_{BR}$.

- By carefully analyzing the order in which computation must occur (in compiler lingo: by performing a dependence analysis), we can identify five states that can be maintained at the top of the loop, by deleting subexpressions from the PME. These are called loop invariants. There are five for LU factorization:

$\left(\begin{array}{c c} A_{TL} & A_{TR} \\ \hline A_{BL} & A_{BR} \end{array} \right) = \left(\begin{array}{c c} L \setminus U_{TL} & \hat{A}_{TR} \\ \hline \hat{A}_{BL} & \hat{A}_{BR} \end{array} \right)$ Invariant 1	$\left(\begin{array}{c c} A_{TL} & A_{TR} \\ \hline A_{BL} & A_{BR} \end{array} \right) = \left(\begin{array}{c c} L \setminus U_{TL} & U_{TR} \\ \hline \hat{A}_{BL} & \hat{A}_{BR} \end{array} \right)$ Invariant 2
$\left(\begin{array}{c c} A_{TL} & A_{TR} \\ \hline A_{BL} & A_{BR} \end{array} \right) = \left(\begin{array}{c c} L \setminus U_{TL} & \hat{A}_{TR} \\ \hline L_{BL} & \hat{A}_{BR} \end{array} \right)$ Invariant 3	$\left(\begin{array}{c c} A_{TL} & A_{TR} \\ \hline A_{BL} & A_{BR} \end{array} \right) = \left(\begin{array}{c c} L \setminus U_{TL} & U_{TR} \\ \hline L_{BL} & \hat{A}_{BR} \end{array} \right)$ Invariant 4
$\left(\begin{array}{c c} A_{TL} & A_{TR} \\ \hline A_{BL} & A_{BR} \end{array} \right) = \left(\begin{array}{c c} L \setminus U_{TL} & U_{TR} \\ \hline L_{BL} & \hat{A}_{BR} - L_{BL}U_{TR} \end{array} \right)$ Invariant 5	

- Key to figuring out what updates must occur in the loop for each of the variants is to look at how the matrices are repartitioned at the top and bottom of the loop body.

For each of the five algorithms for LU factorization, we will derive the loop invariant, and then derive the algorithm from the loop invariant.

5.5.1.1 Variant 1: Bordered algorithm ¶

Consider the loop invariant:

$$\left(\begin{array}{c|c} A_{TL} & A_{TR} \\ \hline A_{BL} & A_{BR} \end{array} \right) = \left(\begin{array}{c|c} L \setminus U_{TL} & \hat{A}_{TR} \\ \hline \hat{A}_{BL} & \hat{A}_{BR} \end{array} \right) \wedge L_{TL}U_{TL} = \hat{A}_{TL},$$

meaning that the leading principal submatrix A_{TL} has been overwritten with its LU factorization, and the remainder of the matrix has not yet been touched.

At the top of the loop, after repartitioning, A then contains

$$\left(\begin{array}{c|cc} A_{00} & a_{01} & A_{02} \\ \hline a_{10}^T & \alpha_{11} & a_{12}^T \\ \hline A_{20} & a_{21} & A_{22} \end{array} \right) = \left(\begin{array}{c|cc} L \setminus U_{00} & \hat{a}_{01} & \hat{A}_{02} \\ \hline \hat{a}_{10}^T & \hat{\alpha}_{11} & \hat{a}_{12}^T \\ \hline \hat{A}_{20} & \hat{a}_{21} & \hat{A}_{22} \end{array} \right) \wedge L_{00}U_{00} = \hat{A}_{00}$$

while after updating A it must contain

$$\left(\begin{array}{c|cc} A_{00} & a_{01} & A_{02} \\ \hline a_{10}^T & \alpha_{11} & a_{12}^T \\ \hline A_{20} & a_{21} & A_{22} \end{array} \right) = \left(\begin{array}{c|cc} L \setminus U_{00} & u_{01} & \hat{A}_{02} \\ \hline l_{10}^T & v_{11} & \hat{a}_{12}^T \\ \hline \hat{A}_{20} & \hat{a}_{21} & \hat{A}_{22} \end{array} \right) \\ \wedge \underbrace{\left(\begin{array}{cc} L_{00} & 0 \\ l_{10}^T & 1 \end{array} \right) \left(\begin{array}{cc} U_{00} & u_{01} \\ 0 & v_{11} \end{array} \right)}_{\begin{array}{ll} L_{00}U_{00} = \hat{A}_{00} & L_{00}u_{01} = \hat{a}_{01} \\ l_{10}^T U_{00} = \hat{a}_{10}^T & l_{10}^T u_{01} + v_{11} = \hat{\alpha}_{11} \end{array}} = \left(\begin{array}{cc} \hat{A}_{00} & \hat{a}_{01} \\ \hat{a}_{10}^T & \hat{\alpha}_{11} \end{array} \right)$$

for the loop invariant to again hold after the iteration. Here the entries in red are known (in addition to the ones marked with a "hat") and the entries in blue are to be computed. With this, we can compute the desired parts of L and U :

- Solve $L_{00}u_{01} = a_{01}$, overwriting a_{01} with the result. (Notice that $a_{01} = \hat{a}_{01}$ before this update.)
- Solve $l_{10}^T U_{00} = a_{10}^T$ (or, equivalently, $U_{00}^T (l_{10}^T)^T = (a_{10}^T)^T$ for l_{10}^T), overwriting a_{10}^T with the result. (Notice that $a_{10}^T = \hat{a}_{10}^T$ before this update.)
- Update $\alpha_{11} := v_{11} = \alpha_{11} - l_{10}^T u_{01}$. (Notice that by this computation, $a_{10}^T = l_{10}^T$ and $a_{01} = u_{01}$.)

The resulting algorithm is captured in [Figure 5.5.1.3](#).

$A = \text{LU-var1}(A)$

$A \rightarrow \left(\begin{array}{c|c} A_{TL} & A_{TR} \\ \hline A_{BL} & A_{BR} \end{array} \right)$

A_{TL} is 0×0

while $n(A_{TL}) < n(A)$

$$\left(\begin{array}{c|c} A_{TL} & A_{TR} \\ \hline A_{BL} & A_{BR} \end{array} \right) \rightarrow \left(\begin{array}{c|cc} A_{00} & a_{01} & A_{02} \\ \hline a_{10}^T & \alpha_{11} & a_{12}^T \\ \hline A_{20} & a_{21} & A_{22} \end{array} \right)$$

Solve $L_{00}u_{01} = a_{01}$ overwriting a_{01} with the result

Solve $l_{10}^T U_{00} = a_{10}^T$ overwriting a_{10}^T with the result

$\alpha_{11} := v_{11} = \alpha_{11} - a_{10}^T a_{01}$

$$\left(\begin{array}{c|c} A_{TL} & A_{TR} \\ \hline A_{BL} & A_{BR} \end{array} \right) \leftarrow \left(\begin{array}{c|cc} A_{00} & a_{01} & A_{02} \\ \hline a_{10}^T & \alpha_{11} & a_{12}^T \\ \hline A_{20} & a_{21} & A_{22} \end{array} \right)$$

endwhile

Figure 5.5.1.3. Variant 1 (bordered) LU factorization algorithm. Here A_{00} stores $L \setminus U_{00}$.

Homework 5.5.1.1. If A is $n \times n$, show the cost of Variant 1 is approximately $\frac{2}{3}n^3$.
[▶ Solution](#)

5.5.1.2 Variant 2: Up-looking algorithm ¶

Consider next the loop invariant:

$$\left(\begin{array}{c|c} A_{TL} & A_{TR} \\ \hline A_{BL} & A_{BR} \end{array} \right) = \left(\begin{array}{c|c} L \setminus U_{TL} & U_{TR} \\ \hline \hat{A}_{BL} & \hat{A}_{BR} \end{array} \right) \wedge L_{TL}U_{TL} = \hat{A}_{TL} \mid L_{TL}U_{TR} = \hat{A}_{TR}$$

meaning that the leading principal submatrix A_{TL} has been overwritten with its LU factorization and U_{TR} has overwritten A_{TR} .

At the top of the loop, after repartitioning, A then contains

$$\left(\begin{array}{c|cc} A_{00} & a_{01} & A_{02} \\ \hline a_{10}^T & \alpha_{11} & a_{12}^T \\ \hline \hat{A}_{20} & \hat{a}_{21} & \hat{A}_{22} \end{array} \right) = \left(\begin{array}{c|cc} L \setminus U_{00} & u_{01} & U_{02} \\ \hline \hat{a}_{10}^T & \hat{\alpha}_{11} & \hat{a}_{12}^T \\ \hline \hat{A}_{20} & \hat{a}_{21} & \hat{A}_{22} \end{array} \right) \\ \wedge \underbrace{L_{00}U_{00} = \hat{A}_{00} \mid L_{00} \begin{pmatrix} u_{01} & U_{02} \end{pmatrix} = \begin{pmatrix} \hat{a}_{01} & \hat{A}_{02} \end{pmatrix}}_{\begin{array}{l} L_{00}U_{00} = \hat{A}_{00} \mid L_{00}u_{01} = \hat{a}_{01} \quad L_{00}U_{02} = \hat{A}_{02} \end{array}}$$

while after updating A it must contain

$$\left(\begin{array}{c|cc} A_{00} & a_{01} & A_{02} \\ \hline a_{10}^T & \alpha_{11} & a_{12}^T \\ \hline A_{20} & a_{21} & A_{22} \end{array} \right) = \left(\begin{array}{c|cc} L \setminus U_{00} & u_{01} & U_{02} \\ \hline l_{10}^T & v_{11} & u_{12}^T \\ \hline \hat{A}_{20} & \hat{a}_{21} & \hat{A}_{22} \end{array} \right) \\ \wedge \underbrace{\begin{pmatrix} L_{00} & 0 \\ l_{10}^T & 1 \end{pmatrix} \begin{pmatrix} U_{00} & u_{01} \\ 0 & v_{11} \end{pmatrix} = \begin{pmatrix} \hat{A}_{00} & \hat{a}_{01} \\ \hat{a}_{10}^T & \hat{\alpha}_{11} \end{pmatrix} \mid \begin{pmatrix} L_{00} & 0 \\ l_{10}^T & 1 \end{pmatrix} \begin{pmatrix} U_{02} \\ u_{12}^T \end{pmatrix} = \begin{pmatrix} \hat{A}_{00} \\ \hat{a}_{12}^T \end{pmatrix}}_{\begin{array}{l} L_{00}U_{00} = \hat{A}_{00} \quad L_{00}u_{01} = \hat{a}_{01} \mid L_{00}U_{02} = \hat{A}_{02} \\ l_{10}^T U_{00} = \hat{a}_{10}^T \quad l_{10}^T u_{01} + v_{11} = \hat{\alpha}_{11} \mid l_{10}^T U_{02} + u_{12}^T = \hat{a}_{12}^T \end{array}}$$

for the loop invariant to again hold after the iteration. Here, again, the entries in red are known (in addition to the ones marked with a "hat") and the entries in blue are to be computed. With this, we can compute the desired parts of L and U :

- Solve $l_{10}^T U_{00} = a_{10}^T$, overwriting a_{10}^T with the result.
- Update $\alpha_{11} := v_{11} = \alpha_{11} - l_{10}^T u_{01} = \alpha_{11} - a_{10}^T a_{01}$.
- Update $a_{12}^T := u_{12}^T = a_{12}^T - l_{10}^T U_{02} = a_{12}^T - a_{10}^T A_{02}$.

The resulting algorithm is captured in [Figure 5.5.1.4](#).

```

A = LU-var2(A)

A → ( ATL | ATR
      ABL | ABR )

ATL is 0 × 0
while n(ATL) < n(A)

    ( ATL | ATR
      ABL | ABR ) → ( A00 | a01 A02
                       a10T | α11 a12T
                       A20 | a21 A22 )

    Solve l10TU00 = a10T, overwriting a10T with the result
    α11 := v11 = α11 - a10Ta01
    a12T := u12T = a12T - l10TU02

    ( ATL | ATR
      ABL | ABR ) ← ( A00 a01 | A02
                       a10T α11 | a12T
                       A20 a21 | A22 )

endwhile

```

Figure 5.5.1.4. Variant 2 (up-looking) LU factorization algorithm. Here A_{00} stores $L \setminus U_{00}$.

Homework 5.5.1.2. If A is $n \times n$, show the cost of Variant 2 is approximately $\frac{2}{3}n^3$.
[▶ Solution](#)

During the k th iteration, A_{00} is $k \times k$, for $k = 0, \dots, n-1$. Then the (approximate) cost of each of the steps is given by

- Solve $l_{10}^T U_{00} = a_{10}^T$, overwriting a_{10}^T with the result. Approximate cost: k^2 flops.
- Update $\alpha_{11} := v_{11} = \alpha_{11} - l_{10}^T u_{01} = \alpha_{11} - a_{10}^T a_{01}$. Approximate cost: $2k$ flops.
- Update $a_{12}^T := u_{12}^T = a_{12}^T - l_{10}^T U_{02} = a_{12}^T - a_{10}^T A_{02}$. Approximate cost: $2k(n-k-1)$ flops.

Thus, the total cost is approximately given by

$$\begin{aligned}
& \sum_{k=0}^{n-1} (k^2 + 2k + 2k(n-k-1)) \\
&= < \text{simplify} > \\
& \sum_{k=0}^{n-1} (2kn - k^2) \\
&= < \text{algebra} > \\
& 2n \sum_{k=0}^{n-1} k - \sum_{k=0}^{n-1} k^2 \\
& \approx < \sum_{k=0}^{n-1} k \approx n^2/2; \sum_{k=0}^{n-1} k^2 \approx k^3/3 > \\
& n^3 - \frac{n^3}{3} \\
&= < \text{simplify} > \\
& \frac{2}{3}n^3.
\end{aligned}$$

5.5.1.3 Variant 3: Left-looking algorithm ¶

Consider the loop invariant:

$$\left(\begin{array}{c|c} A_{TL} & A_{TR} \\ \hline A_{BL} & A_{BR} \end{array} \right) = \left(\begin{array}{c|c} \textcolor{red}{L} \backslash U_{TL} & \hat{A}_{TR} \\ \hline \textcolor{red}{L}_{BL} & \hat{A}_{BR} \end{array} \right) \wedge \frac{L_{TL} U_{TL} = \hat{A}_{TL}}{L_{BL} U_{TL} = \hat{A}_{BL}}$$

At the top of the loop, after repartitioning, A then contains

$$\left(\begin{array}{c|cc} A_{00} & a_{01} & A_{02} \\ \hline a_{10}^T & \alpha_{11} & a_{12}^T \\ \hline A_{20} & a_{21} & A_{22} \end{array} \right) = \left(\begin{array}{c|cc} \textcolor{red}{L} \backslash U_{00} & \hat{a}_{01} & \hat{A}_{02} \\ \hline \textcolor{red}{l}_{10}^T & \hat{\alpha}_{11} & \hat{a}_{12}^T \\ \hline \textcolor{red}{L}_{20} & \hat{a}_{21} & \hat{A}_{22} \end{array} \right) \wedge \frac{L_{00} U_{00} = \hat{A}_{00}}{\textcolor{red}{l}_{10}^T U_{00} = \hat{a}_{10}^T} \quad \frac{L_{20} U_{00} = \hat{A}_{20}}{\textcolor{red}{l}_{10}^T U_{00} = \hat{a}_{10}^T}$$

while after updating A it must contain

$$\begin{aligned}
& \left(\begin{array}{c|cc} A_{00} & a_{01} & A_{02} \\ \hline a_{10}^T & \alpha_{11} & a_{12}^T \\ \hline A_{20} & a_{21} & A_{22} \end{array} \right) = \left(\begin{array}{c|cc} \textcolor{red}{L} \backslash U_{00} & u_{01} & \hat{A}_{02} \\ \hline \textcolor{red}{l}_{10}^T & v_{11} & \hat{a}_{12}^T \\ \hline \textcolor{red}{L}_{20} & l_{21} & \hat{A}_{22} \end{array} \right) \\
& \wedge \frac{\left(\begin{array}{cc} L_{00} & 0 \\ \hline l_{10}^T & 1 \end{array} \right) \left(\begin{array}{cc} U_{00} & u_{01} \\ \hline 0 & v_{11} \end{array} \right) = \left(\begin{array}{cc} \hat{A}_{00} & \hat{a}_{01} \\ \hline \hat{a}_{10}^T & \hat{\alpha}_{11} \end{array} \right)}{\left(\begin{array}{cc} L_{20} & l_{21} \end{array} \right) \left(\begin{array}{cc} U_{00} & u_{01} \\ \hline 0 & v_{11} \end{array} \right) = \left(\begin{array}{cc} \hat{A}_{20} & \hat{a}_{21} \end{array} \right)} \\
& \frac{L_{00} U_{00} = \hat{A}_{00} \quad L_{00} u_{01} = \hat{a}_{01}}{\textcolor{red}{l}_{10}^T U_{00} = \hat{a}_{10}^T \quad \textcolor{red}{l}_{10}^T u_{01} + v_{11} = \hat{\alpha}_{11}} \\
& \frac{L_{20} U_{00} = \hat{A}_{20} \quad L_{20} u_{01} + l_{21} v_{11} = \hat{a}_{21}}{}
\end{aligned}$$

for the loop invariant to again hold after the iteration. With this, we can compute the desired parts of L and U :

- Solve $L_{00} u_{01} = a_{01}$, overwriting a_{01} with the result.
- Update $\alpha_{11} := v_{11} = \alpha_{11} - l_{10}^T u_{01} = \alpha_{11} - a_{10}^T a_{01}$.
- Update $a_{21} := l_{21} = (a_{21} - L_{20} u_{01}) / v_{11} = (a_{21} - A_{20} a_{01}) / \alpha_{11}$.

The resulting algorithm is captured in [Figure 5.5.1.5](#).

$A = \text{LU-var3}(A)$
$A \rightarrow \left(\begin{array}{c c} A_{TL} & A_{TR} \\ \hline A_{BL} & A_{BR} \end{array} \right)$
A_{TL} is 0×0
while $n(A_{TL}) < n(A)$
$\left(\begin{array}{c c} A_{TL} & A_{TR} \\ \hline A_{BL} & A_{BR} \end{array} \right) \rightarrow \left(\begin{array}{c cc} A_{00} & a_{01} & A_{02} \\ \hline a_{10}^T & \alpha_{11} & a_{12}^T \\ \hline A_{20} & a_{21} & A_{22} \end{array} \right)$
Solve $L_{00} u_{01} = a_{01}$ overwriting a_{01} with the result
$\alpha_{11} := v_{11} = \alpha_{11} - a_{10}^T a_{01}$
$a_{21} := l_{21} = (a_{21} - A_{20} a_{01}) / \alpha_{11}$
$\left(\begin{array}{c c} A_{TL} & A_{TR} \\ \hline A_{BL} & A_{BR} \end{array} \right) \leftarrow \left(\begin{array}{c cc} A_{00} & a_{01} & A_{02} \\ \hline a_{10}^T & \alpha_{11} & a_{12}^T \\ \hline A_{20} & a_{21} & A_{22} \end{array} \right)$
endwhile

Figure 5.5.1.5. Variant 3 (left-looking) LU factorization algorithm. Here A_{00} stores $L \backslash U_{00}$.

Homework 5.5.1.3. If A is $n \times n$, show the cost of Variant 3 is approximately $\frac{2}{3}n^3$.

▼ [Solution](#)

During the k th iteration, A_{00} is $k \times k$, for $k = 0, \dots, n-1$. Then the (approximate) cost of each of the steps is given by

- Solve $L_{00} u_{01} = a_{01}$, overwriting a_{01} with the result. Approximate cost: k^2 flops.
- Update $\alpha_{11} := v_{11} = \alpha_{11} - l_{10}^T u_{01} = \alpha_{11} - a_{10}^T a_{01}$. Approximate cost: $2k$ flops.
- Update $a_{21} := l_{21} = (a_{21} - L_{20} u_{01}) / v_{11} = (a_{21} - A_{20} a_{01}) / \alpha_{11}$. Approximate cost: $2(n-k-1)$ flops.

Thus, the total cost is approximately given by

$$\begin{aligned}
& \sum_{k=0}^{n-1} (k^2 + 2k + 2k(n-k-1)) \\
& = < \text{simplify} > \\
& \sum_{k=0}^{n-1} (2kn - k^2) \\
& = < \text{algebra} > \\
& 2n \sum_{k=0}^{n-1} k - \sum_{k=0}^{n-1} k^2 \\
& \approx < \sum_{k=0}^{n-1} k \approx n^2/2; \sum_{k=0}^{n-1} k^2 \approx k^3/3 > \\
& n^3 - \frac{n^3}{3} \\
& = < \text{simplify} > \\
& \frac{2}{3}n^3.
\end{aligned}$$

5.5.1.4 Variant 4: Crout variant ¶

Consider next the loop invariant:

$$\left(\begin{array}{c|c} A_{TL} & A_{TR} \\ \hline A_{BL} & A_{BR} \end{array} \right) = \left(\begin{array}{c|c} L \backslash U_{TL} & U_{TR} \\ \hline L_{BL} & \hat{A}_{BR} \end{array} \right) \wedge \frac{L_{TL}U_{TL} = \hat{A}_{TL} \mid L_{TL}U_{TR} = \hat{A}_{TR}}{L_{BL}U_{TL} = \hat{A}_{BL}}$$

At the top of the loop, after repartitioning, A then contains

$$\left(\begin{array}{c|cc} A_{00} & a_{01} & A_{02} \\ \hline a_{10}^T & \alpha_{11} & a_{12}^T \\ A_{20} & a_{21} & A_{22} \end{array} \right) = \left(\begin{array}{c|cc} L \backslash U_{00} & u_{01} & U_{02} \\ \hline l_{10}^T & \hat{\alpha}_{11} & \hat{a}_{12}^T \\ L_{20} & \hat{a}_{21} & \hat{A}_{22} \end{array} \right)$$

$$\wedge \frac{L_{00}U_{00} = \hat{A}_{00} \mid L_{00}u_{01} = \hat{a}_{01} \mid L_{00}U_{02} = \hat{A}_{02}}{l_{10}^T U_{00} = \hat{a}_{10}^T \mid L_{20}U_{00} = \hat{A}_{20}}$$

while after updating A it must contain

$$\left(\begin{array}{c|cc} A_{00} & a_{01} & A_{02} \\ \hline a_{10}^T & \alpha_{11} & a_{12}^T \\ A_{20} & a_{21} & A_{22} \end{array} \right) = \left(\begin{array}{c|cc} L \backslash U_{00} & u_{01} & U_{02} \\ \hline l_{10}^T & v_{11} & u_{12}^T \\ L_{20} & l_{21} & \hat{A}_{22} \end{array} \right)$$

$$\wedge \frac{\left(\begin{array}{cc} L_{00} & 0 \\ l_{10}^T & 1 \end{array} \right) \left(\begin{array}{cc} U_{00} & u_{01} \\ 0 & v_{11} \end{array} \right) = \left(\begin{array}{cc} \hat{A}_{00} & \hat{a}_{01} \\ \hat{a}_{10}^T & \hat{\alpha}_{11} \end{array} \right) \mid \left(\begin{array}{cc} L_{00} & 0 \\ l_{10}^T & 1 \end{array} \right) \left(\begin{array}{c} U_{02} \\ u_{12}^T \end{array} \right) = \left(\begin{array}{c} \hat{A}_{02} \\ \hat{a}_{12}^T \end{array} \right)}{\left(\begin{array}{cc} L_{20} & l_{21} \end{array} \right) \left(\begin{array}{cc} U_{00} & u_{01} \\ 0 & v_{11} \end{array} \right) = \left(\begin{array}{cc} \hat{A}_{20} & \hat{a}_{21} \end{array} \right)}$$

$$\frac{L_{00}U_{00} = \hat{A}_{00} \mid L_{00}u_{01} = \hat{a}_{01} \mid L_{00}U_{02} = \hat{A}_{02}}{l_{10}^T U_{00} = \hat{a}_{10}^T \mid l_{10}^T u_{01} + v_{11} = \hat{\alpha}_{11} \mid l_{10}^T U_{02} + u_{12}^T = \hat{a}_{12}^T}$$

$$\frac{L_{20}U_{00} = \hat{A}_{20} \mid L_{20}u_{01} + l_{21}v_{11} = \hat{a}_{21}}{}$$

for the loop invariant to again hold after the iteration. With this, we can compute the desired parts of L and U :

- Update $\alpha_{11} := v_{11} = \alpha_{11} - l_{10}^T u_{01} = \alpha_{11} - a_{10}^T a_{01}$.
- Update $a_{12}^T := u_{12}^T = a_{12}^T - l_{10}^T U_{02} = a_{12}^T - a_{10}^T A_{02}$.
- Update $a_{21} := l_{21} = (a_{21} - L_{20}u_{01})/v_{11} = (a_{21} - A_{20}a_{01})/\alpha_{11}$.

The resulting algorithm is captured in [Figure 5.5.1.6](#).

```

A = LU-var4(A)
A → ( ATL | ATR
      ABL | ABR )
ATL is 0 × 0
while n(ATL) < n(A)
    ( ATL | ATR
      ABL | ABR ) → ( A00 | a01 | A02
                       a10T | α11 | a12T
                       A20 | a21 | A22 )
    α11 := v11 = α11 - a10Ta01
    a12T := u12T = a12T - a10TA02
    a21 := l21 = (a21 - A20a01)/α11
    ( ATL | ATR
      ABL | ABR ) ← ( A00 | a01 | A02
                       a10T | α11 | a12T
                       A20 | a21 | A22 )
endwhile

```

Figure 5.5.1.6. Variant 4 (Crout) LU factorization algorithm.

Homework 5.5.1.4. If A is $n \times n$, show the cost of Variant 4 is approximately $\frac{2}{3}n^3$.

▼ [Solution](#)

During the k th iteration, A_{00} is $k \times k$, for $k = 0, \dots, n-1$. Then the (approximate) cost of each of the steps is given by

- Update $\alpha_{11} := v_{11} = \alpha_{11} - l_{10}^T u_{01} = \alpha_{11} - a_{10}^T a_{01}$. Approximate cost: $2k$ flops.
- Update $a_{12}^T := u_{12}^T = a_{12}^T - l_{10}^T U_{02} = a_{12}^T - a_{10}^T A_{02}$. Approximate cost: $2k(n-k-1)$ flops.
- Update $a_{21} := l_{21} = (a_{21} - L_{20}u_{01})/v_{11} = (a_{21} - A_{20}a_{01})/\alpha_{11}$. Approximate cost: $2k(n-k-1) + (n-k-1)$ flops.

Thus, ignoring the $2k$ flops for the dot product and the $n-k-1$ flops for multiplying with $1/\alpha_{11}$ in each iteration, the total cost is approximately given by

$$\begin{aligned}
& \sum_{k=0}^{n-1} 4k(n-k-1) \\
& \approx < \text{remove lower order term} > \sum_{k=0}^{n-1} 4k(n-k) \\
& = < \text{algebra} > \\
& 4n \sum_{k=0}^{n-1} k - 4 \sum_{k=0}^{n-1} k^2 \\
& \approx < \sum_{k=0}^{n-1} k \approx n^2/2; \sum_{k=0}^{n-1} k^2 \approx k^3/3 > \\
& 2n^3 - 4 \frac{n^3}{3} \\
& = < \text{simplify} > \\
& \frac{2}{3}n^3.
\end{aligned}$$

5.5.1.5 Variant 5: Classical Gaussian elimination ¶

Consider final loop invariant:

$$\left(\begin{array}{c|c} A_{TL} & A_{TR} \\ \hline A_{BL} & A_{BR} \end{array} \right) = \left(\begin{array}{c|c} L \backslash U_{TL} & U_{TR} \\ \hline L_{BL} & \hat{A}_{BR} - L_{BL}U_{TR} \end{array} \right) \wedge \frac{L_{TL}U_{TL} = \hat{A}_{TL}}{L_{BL}U_{TL} = \hat{A}_{BL}} \left| \begin{array}{c} L_{TL}U_{TR} = \hat{A}_{TR} \end{array} \right.$$

At the top of the loop, after repartitioning, A then contains

$$\left(\begin{array}{c|c|c} A_{00} & a_{01} & A_{02} \\ \hline a_{10}^T & \alpha_{11} & a_{12}^T \\ \hline A_{20} & a_{21} & A_{22} \end{array} \right) = \left(\begin{array}{c|c|c} L \backslash U_{00} & u_{01} & U_{02} \\ \hline l_{10}^T & \hat{\alpha}_{11} - l_{10}^T u_{01} & \hat{a}_{12} - l_{10}^T U_{02} \\ \hline L_{20} & \hat{a}_{21} - L_{20}u_{01} & \hat{A}_{22} - L_{20}U_{02} \end{array} \right)$$

$$\wedge \frac{L_{00}U_{00} = \hat{A}_{00}}{l_{10}^T U_{00} = \hat{a}_{10}^T} \left| \begin{array}{c} L_{00}u_{01} = \hat{a}_{01} \quad L_{00}U_{02} = \hat{A}_{02} \\ \hline L_{20}U_{00} = \hat{A}_{20} \end{array} \right.$$

while after updating A it must contain

$$\left(\begin{array}{c|c|c} A_{00} & a_{01} & A_{02} \\ \hline a_{10}^T & \alpha_{11} & a_{12}^T \\ \hline A_{20} & a_{21} & A_{22} \end{array} \right) = \left(\begin{array}{c|c|c} L \backslash U_{00} & u_{01} & U_{02} \\ \hline l_{10}^T & v_{11} & u_{12}^T \\ \hline L_{20} & l_{21} & \hat{A}_{22} - l_{21}u_{12}^T \end{array} \right)$$

$$\wedge \frac{\left(\begin{array}{cc} L_{00} & 0 \\ l_{10}^T & 1 \end{array} \right) \left(\begin{array}{cc} U_{00} & u_{01} \\ 0 & v_{11} \end{array} \right) = \left(\begin{array}{cc} \hat{A}_{00} & \hat{a}_{01} \\ \hat{a}_{10}^T & \hat{\alpha}_{11} \end{array} \right) \left| \left(\begin{array}{cc} L_{00} & 0 \\ l_{10}^T & 1 \end{array} \right) \left(\begin{array}{c} U_{02} \\ u_{12}^T \end{array} \right) = \left(\begin{array}{c} \hat{A}_{02} \\ \hat{a}_{12}^T \end{array} \right) \right.}{\left(\begin{array}{cc} L_{20} & l_{21} \end{array} \right) \left(\begin{array}{cc} U_{00} & u_{01} \\ 0 & v_{11} \end{array} \right) = \left(\begin{array}{cc} \hat{A}_{20} & \hat{a}_{21} \end{array} \right)} \left| \begin{array}{c} L_{00}U_{00} = \hat{A}_{00} \quad L_{00}u_{01} = \hat{a}_{01} \quad L_{00}U_{02} = \hat{A}_{02} \\ \hline l_{10}^T U_{00} = \hat{a}_{10}^T \quad l_{10}^T u_{01} + v_{11} = \hat{\alpha}_{11} \quad l_{10}^T U_{02} + u_{12}^T = \hat{a}_{12}^T \\ \hline L_{20}U_{00} = \hat{A}_{20} \quad L_{20}u_{01} + l_{21}v_{11} = \hat{a}_{21} \end{array} \right.$$

for the loop invariant to again hold after the iteration. With this, we can compute the desired parts of L and U :

- $\alpha_{11} := v_{11} = \hat{\alpha}_{11} - l_{10}^T u_{01} = \alpha_{11}$ (no-op).
(α_{11} already equals $\hat{\alpha}_{11} - l_{10}^T u_{01}$.)
- $a_{12}^T := u_{12}^T = \hat{a}_{12}^T - l_{10}^T U_{02} = a_{12}^T$ (no-op).
(a_{12}^T already equals $\hat{a}_{12}^T - l_{10}^T U_{02}$.)
- Update $a_{21} := (\hat{a}_{21} - L_{20}u_{01})/v_{11} = a_{21}/\alpha_{11}$.
(a_{21} already equals $\hat{a}_{21} - L_{20}u_{01}$.)
- Update $A_{22} := \hat{A}_{22} - L_{20}U_{02} - l_{21}u_{12}^T = A_{22} - a_{21}a_{12}^T$.
(A_{22} already equals $\hat{A}_{22} - L_{20}U_{02}$.)

The resulting algorithm is captured in [Figure 5.5.1.7](#).

$$\begin{aligned}
& A = \text{LU-var5}(A) \\
& A \rightarrow \left(\begin{array}{c|c} A_{TL} & A_{TR} \\ \hline A_{BL} & A_{BR} \end{array} \right) \\
& A_{TL} \text{ is } 0 \times 0 \\
& \textbf{while } n(A_{TL}) < n(A) \\
& \quad \left(\begin{array}{c|c} A_{TL} & A_{TR} \\ \hline A_{BL} & A_{BR} \end{array} \right) \rightarrow \left(\begin{array}{c|c|c} A_{00} & a_{01} & A_{02} \\ \hline a_{10}^T & \alpha_{11} & a_{12}^T \\ \hline A_{20} & a_{21} & A_{22} \end{array} \right) \\
& \quad \underline{a_{21} := l_{21} = a_{21}/\alpha_{11}} \\
& \quad \underline{A_{22} := A_{22} - a_{21}a_{12}^T} \\
& \quad \left(\begin{array}{c|c} A_{TL} & A_{TR} \\ \hline A_{BL} & A_{BR} \end{array} \right) \leftarrow \left(\begin{array}{c|c|c} A_{00} & a_{01} & A_{02} \\ \hline a_{10}^T & \alpha_{11} & a_{12}^T \\ \hline A_{20} & a_{21} & A_{22} \end{array} \right) \\
& \textbf{endwhile}
\end{aligned}$$

Figure 5.5.1.7. Variant 5 (classical Gaussian elimination) LU factorization algorithm.

Homework 5.5.1.5. If A is $n \times n$, show the cost of Variant 5 is approximately $\frac{2}{3}n^3$.

▼ [Solution](#)

During the k th iteration, A_{00} is $k \times k$, for $k = 0, \dots, n-1$. Then the (approximate) cost of each of the steps is given by

- Update $a_{21} := l_{21} = a_{21}/\alpha_{11}$. Approximate cost: k flops.
- Update $A_{22} := A_{22} - l_{21}u_{12}^T = A_{22} - a_{21}a_{12}^T$. Approximate cost: $2(n-k-1)(n-k-1)$ flops.

Thus, ignoring $n - k - 1$ flops for multiplying with $1/\alpha_{11}$ in each iteration, the total cost is approximately given by

$$\begin{aligned} & \sum_{k=0}^{n-1} 2(n-k-1)^2 \\ &= < \text{change of variable } j = n - k - 1 > \\ & 2 \sum_{j=0}^{n-1} j^2 \\ & \approx < \sum_{k=0}^{n-1} k^2 \approx k^3/3 > \\ & 2 \frac{n^3}{3} \end{aligned}$$

5.5.1.6 Discussion ¶

Remark 5.5.1.8. For a discussion of the different LU factorization algorithms that also gives a historic perspective, we recommend "Matrix Algorithms Volume 1" by G.W. Stewart [\[38\]](#).